

Лабораторная работа №6

Рекурсивные алгоритмы. Простые сортировки

Цель работы:

1. Освоить принципы рекурсии: прямой и косвенной, базовый случай и шаг рекурсии
2. Научиться применять рекурсию для решения практических задач (факториал, числа Фибоначчи, Ханойские башни, обход лабиринта)
3. Реализовать и сравнить простые алгоритмы сортировки: пузырьковую, вставками, выбором
4. Проанализировать сложность рекурсивных алгоритмов и простых сортировок
5. Сравнить рекурсивную и итеративную реализации одних и тех же алгоритмов

Теоретическое обоснование

1. Что такое рекурсия?

Рекурсия - это метод программирования, при котором функция вызывает саму себя для решения подзадач меньшего размера.

Основные компоненты рекурсивной функции:

Компонент	Описание	Пример для факториала
Базовый случай	Условие остановки рекурсии	if (n == 0) return 1;
Шаг рекурсии	Вызов функции с меньшими параметрами	return n * factorial(n - 1);

Визуализация рекурсии (факториал 5):

factorial(5)

→ 5 * factorial(4)

→ 4 * factorial(3)

→ 3 * factorial(2)

→ 2 * factorial(1)

→ 1 * factorial(0)

→ 1 (базовый случай)

← 1 * 1 = 1

$$\begin{aligned} &\leftarrow 2 * 1 = 2 \\ &\leftarrow 3 * 2 = 6 \\ &\leftarrow 4 * 6 = 24 \\ &\leftarrow 5 * 24 = 120 \end{aligned}$$

2. Виды рекурсии

Вид	Описание	Пример
Прямая рекурсия	Функция вызывает саму себя	factorial(n) вызывает factorial(n-1)
Косвенная рекурсия	Функция А вызывает В, В вызывает А	even(n) → odd(n-1), odd(n) → even(n-1)
Хвостовая рекурсия	Рекурсивный вызов - последняя операция	sum(n, acc) с аккумулятором

3. Классические рекурсивные задачи

Факториал

```
// Рекурсивно
int factorial(int n) {
    if (n <= 1) return 1;      // базовый случай
    return n * factorial(n - 1); // шаг рекурсии
}
```

```
// Итеративно
int factorialIter(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}
```

Числа Фибоначчи

```
// Рекурсивно (экспоненциальная сложность O(2^n))
int fibRecursive(int n) {
    if (n <= 1) return n;
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}
```

```

    }

    // Итеративно (линейная сложность O(n))
    int fibIterative(int n) {
        if (n <= 1) return n;
        int a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
}

```

Ханойские башни

```

void hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        cout << "Переместить диск 1 с " << from << " на " << to << endl;
        return;
    }
    hanoi(n - 1, from, aux, to);
    cout << "Переместить диск " << n << " с " << from << " на " << to <<
endl
    hanoi(n - 1, aux, to, from);
}

```

4. Простые сортировки Сравнение сложности

Алгоритм	Лучший случай	Средний случай	Худший случай	Память	Устойчивость
Пузырьковая	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Да
Вставками	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Да
Выбором	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Нет

Пузырьковая сортировка (Bubble Sort)

```

void bubbleSort(int arr[], int n) {

```

```

for (int i = 0; i < n - 1; i++) {
    bool swapped = false;
    for (int j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
            swapped = true;
        }
    }
    if (!swapped) break; // ОПТИМИЗАЦИЯ
}
}

```

Сортировка вставками (Insertion Sort)

```

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

```

Сортировка выбором (Selection Sort)

```

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            swap(arr[i], arr[minIndex]);
        }
    }
}

```

Примеры выполнения практической части

Пример 1. Рекурсивный обход лабиринта

Задача: Найти путь от старта (S) до финиша (F) в лабиринте с помощью рекурсивного поиска с возвратом (backtracking).

```
#include <iostream>
#include <vector>

using namespace std;

const int N = 5;
char maze[N][N] = {
    {'S', '!', '#', '!', '!'},
    {'#', '!', '#', '!', '#'},
    {'!', '!', '!', '#', '!'},
    {'#', '#', '!', '!', '!'},
    {'!', '!', '#', '!', 'F'}
};

bool visited[N][N] = {false};

bool findPath(int x, int y) {
    // Проверка выхода за границы
    if (x < 0 || x >= N || y < 0 || y >= N) return false;
    // Проверка стены или посещённой клетки
    if (maze[x][y] == '#' || visited[x][y]) return false;
    // Проверка достижения финиша
    if (maze[x][y] == 'F') {
        maze[x][y] = '*';
        return true;
    }

    visited[x][y] = true;

    // Рекурсивный поиск в 4 направлениях
    if (findPath(x + 1, y) || // вниз
        findPath(x - 1, y) || // вверх
        findPath(x, y + 1) || // вправо
        findPath(x, y - 1)) { // влево
        if (maze[x][y] != 'S') maze[x][y] = '*';
        return true;
    }
}
```

```

        return false;
    }

int main() {
    int startX = 0, startY = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (maze[i][j] == 'S') {
                startX = i;
                startY = j;
            }
        }
    }

    if (findPath(startX, startY)) {
        cout << "Путь найден!" << endl;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                cout << maze[i][j] << " ";
            }
            cout << endl;
        }
    } else {
        cout << "Путь не найден" << endl;
    }

    return 0;
}

```

Ожидаемый вывод:

Путь найден!

```

S * # . .
# * # . #
* * * # .
# # * * *
. . # . F

```

Пример 2. Сравнение сортировок

Задача: Реализовать три простые сортировки и сравнить их производительность.

```
#include <iostream>
```

```

#include <chrono>
#include <algorithm>

using namespace std;
using namespace chrono;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) swap(arr[i], arr[minIndex]);
    }
}

```

```

long long measureTime(void (*sortFunc)(int[], int), int arr[], int n) {
    int* copy = new int[n];
    for (int i = 0; i < n; i++) copy[i] = arr[i];

    auto start = high_resolution_clock::now();
    sortFunc(copy, n);
    auto end = high_resolution_clock::now();

    delete[] copy;
    return duration_cast<microseconds>(end - start).count();
}

int main() {
    const int N = 10000;
    int* arr = new int[N];

    // Заполняем случайными числами
    for (int i = 0; i < N; i++) arr[i] = rand() % N;

    cout << "Сравнение производительности (N = " << N << "):" << endl;
    cout << "Пузырьковая: " << measureTime(bubbleSort, arr, N) << " мкс"
<< endl;
    cout << "Вставками: " << measureTime(insertionSort, arr, N) << " мкс"
<< endl;
    cout << "Выбором: " << measureTime(selectionSort, arr, N) << " мкс"
<< endl;

    delete[] arr;
    return 0;
}

```

Индивидуальные задания

Часть 1. Рекурсивные алгоритмы

Каждый вариант содержит 4 рекурсивные задачи:

№ вар	Задача 1	Задача 2	Задача 3	Задача 4
1	Факториал числа	Числа Фибоначчи	Сумма цифр числа	Степень числа (a^n)

№ вар	Задача 1	Задача 2	Задача 3	Задача 4
2	Факториал числа	Числа Фибоначчи	Количество цифр	Бинарный поиск
3	Факториал числа	Числа Фибоначчи	Переворот строки	Проверка палиндрома
4	Факториал числа	Числа Фибоначчи	НОД (алгоритм Евклида)	Ханойские башни (3 диска)
5	Факториал числа	Числа Фибоначчи	Сумма элементов массива	Ханойские башни (4 диска)
6	Факториал числа	Числа Фибоначчи	Найти максимум в массиве	Ханойские башни (5 дисков)
7	Факториал числа	Треугольные числа	Произведение цифр	Перебор всех перестановок
8	Факториал числа	Число сочетаний $C(n,k)$	Разложение на слагаемые	Генерация всех подмножеств
9	Факториал числа	Числа Фибоначчи	Обход лабиринта (4×4)	Задача о ходе коня
10	Факториал числа	Числа Фибоначчи	Обход лабиринта (5×5)	Задача о ферзях (4×4)
11	Факториал числа	Числа Фибоначчи	Обход лабиринта (6×6)	Задача о ферзях (5×5)
12	Факториал числа	Числа Фибоначчи	Проверка сбалансированности скобок	Генерация всех комбинаций

№ вар.	Задача 1	Задача 2	Задача 3	Задача 4
13	Факториал числа	Числа Фибоначчи	Вычисление определителя (2×2)	Ханойские башни (с подсчётом ходов)
14	Факториал числа	Числа Фибоначчи	Вычисление определителя (3×3)	Обход лабиринта (с поиском кратчайшего пути)
15	Факториал числа	Числа Фибоначчи	Алгоритм быстрого возведения в степень	Ханойские башни (с визуализацией)

Часть 2. Простые сортировки

Каждый вариант содержит задание реализовать сортировки и выполнить анализ:

№ вар.	Сортировки для реализации	Дополнительное задание
1	Пузырьковая, вставками, выбором	Сравнить производительность на случайном массиве
2	Пузырьковая, вставками, выбором	Сравнить на почти отсортированном массиве
3	Пузырьковая, вставками, выбором	Сравнить на массиве, отсортированном в обратном порядке
4	Пузырьковая, вставками	Подсчитать количество сравнений и обменов

№ вар.	Сортировки для реализации	Дополнительное задание
5	Пузырьковая, выбором	Подсчитать количество сравнений и обменов
6	Вставками, выбором	Подсчитать количество сравнений и обменов
7	Пузырьковая, вставками, выбором	Определить лучшую сортировку для разных типов данных
8	Пузырьковая, вставками, выбором	Реализовать устойчивую версию сортировки выбором
9	Пузырьковая, вставками, выбором	Реализовать сортировку вставками для связного списка
10	Пузырьковая, вставками, выбором	Реализовать сортировку выбором для связного списка
11	Пузырьковая, вставками, выбором	Реализовать рекурсивную версию пузырьковой сортировки
12	Пузырьковая, вставками, выбором	Реализовать рекурсивную версию сортировки вставками
13	Пузырьковая, вставками, выбором	Реализовать рекурсивную версию сортировки выбором
14	Пузырьковая, вставками, выбором	Сравнить производительность с std::sort
15	Пузырьковая, вставками, выбором	Визуализировать процесс сортировки (пошаговый вывод)

Контрольные вопросы

1. Что такое рекурсия? Из каких компонентов состоит рекурсивная функция?
2. Что такое базовый случай рекурсии? Почему он важен?
3. Чем отличается прямая рекурсия от косвенной?
4. Что такое хвостовая рекурсия? В чём её преимущество?

5. Какова временная сложность рекурсивного вычисления чисел Фибоначчи? Почему?
6. Как можно оптимизировать рекурсивное вычисление Фибоначчи?
7. В чём отличие рекурсивной реализации от итеративной?
8. Какова сложность пузырьковой сортировки? Когда она становится $O(n)$?
9. Чем сортировка вставками отличается от сортировки выбором?
10. Какая из простых сортировок является устойчивой? Что это значит?
11. Какая из простых сортировок работает лучше на почти отсортированных данных?
12. Как реализовать пузырьковую сортировку с оптимизацией?
13. Что такое рекурсивный поиск с возвратом (backtracking)?
14. Как рекурсивно обойти лабиринт?
15. Какова сложность Ханойских башен? Сколько ходов нужно для перемещения n дисков?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (рекурсия, простые сортировки)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода)

Анализ сложности рекурсивных алгоритмов

Сравнительная таблица сортировок

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Рекурсивные алгоритмы (4 задачи)	50
1.1	Задача 1 решена корректно	12
1.2	Задача 2 решена корректно	12
1.3	Задача 3 решена корректно	13

№	Критерий	Баллы
1.4	Задача 4 решена корректно	13
2	Простые сортировки	40
2.1	Реализация пузырьковой сортировки	10
2.2	Реализация сортировки вставками	10
2.3	Реализация сортировки выбором	10
2.4	Анализ производительности и сравнение	10
3	Оформление и отчёт	10
3.1	Код отформатирован, есть комментарии	5
3.2	Есть ответы на контрольные вопросы, ссылка на Git	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Отсутствие базового случая в рекурсии (бесконечная рекурсия)	-20
Неверная сложность сортировки	-10
Нет анализа производительности	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60